The *Celestial Emporium of Benevolent Knowledge*, a fictitious taxonomy of animals in *The Analytical Language of John Wilkins*, was said to divide animals into

> *"(a) those that belong to the Emperor, (b) embalmed ones, (c) those that are trained, (d) suckling pigs, (e) mermaids, (f) fabulous ones, (g) stray dogs, (h) those that are included in the present classification, (i) those that tremble as if they are mad, (j) innumerable ones, (k) those drawn with a very fine camelhair brush, (l) others, (m) those that have just broken a flower vase, (n) those that look like flies from a long way off"*
>
> — *Jorge Luis Borges*

ILOs

After this class the student

- understands and is able to explain a modular approach to text classification composed of a probabilistic model and a decision rule;
- understands and is able to explain the need for vector-valued representations of text;
- is able to design hand-crafted representation spaces;
- understands and is able to explain softmax regression;
- knows how to evaluate a text classifier;
- can recognise and explain limitations of the approaches presented.

NLP1

Text Classification

Wilker Aziz



https://nlp-illc.github.io/nlp1/2026/

Fall 2026

Outline

1. Text Classification
   - Basic Example
2. Text Representation
3. Softmax Regression
   - Log-Linear Parameterisation
   - Neural Parameterisation
   - Parameter Estimation
4. Evaluation
5. Limitations

Text Classification

Text classification



An input text is categorised as pertaining to one of K possible classes.[1]

[1] Finite K.



The home team won the league championship on Sunday after its captain scored in the final minute. Thousands of fans celebrated as the players lifted the trophy following a tense 2–1 victory.



An input text is categorised as pertaining to one of K possible classes.[1]

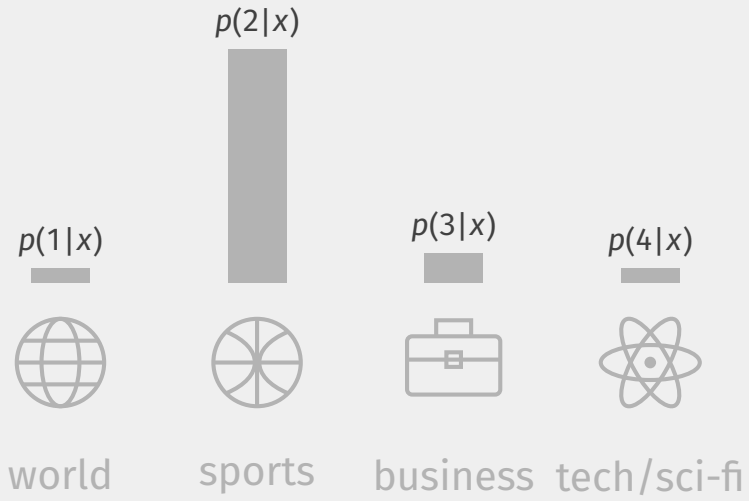
[1] Finite K.

Probabilistic Model

First, we use ideas from ML to parameterise a Categorical probability mass function (pmf),



The home team won the league championship on Sunday after its captain scored in the final minute. Thousands of fans celebrated as the players lifted the trophy following a tense 2–1 victory.



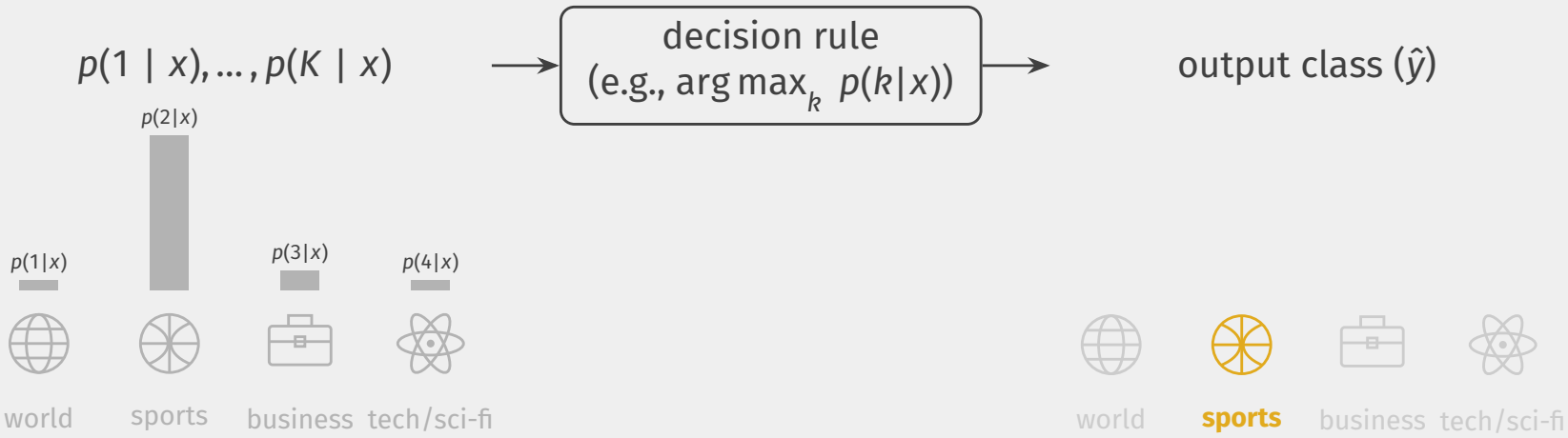
with probability masses predicted conditioned on x.[2]

[2] If A is a Categorical rv with parameters $\boldsymbol{\theta} = (\theta_1, \ldots, \theta_K)^\mathsf{T}$, its pmf is $f_{\boldsymbol{\theta}}(k) = \theta_k$.

Decision Rule

Then, to go from a pmf to a class, for a given input x, we employ a **decision rule**.

In NLP, it is common to use the **mode** of the distribution:



Text Classification

Basic Example

Tabular Conditional Probability Distributions

A tabular CPD simply stores the probabilities of the possible classes, given each of the possible conditions.

Document (x)	world	sports	business	tech/sci-fi
Peace talks resume as leaders agree to meet.	$\frac{1}{1}$	$\frac{0}{1}$	$\frac{0}{1}$	$\frac{0}{1}$
Late goal seals league championship victory.	$\frac{0}{1}$	$\frac{1}{1}$	$\frac{0}{1}$	$\frac{0}{1}$
Company shares rise after record quarterly profits.	$\frac{0}{1}$	$\frac{0}{1}$	$\frac{1}{1}$	$\frac{0}{1}$
Researchers unveil a faster, energy-efficient computer chip.	$\frac{0}{1}$	$\frac{0}{1}$	$\frac{0}{1}$	$\frac{1}{1}$
China and the US announce a new trade agreement.	$\frac{1}{2}$	$\frac{0}{2}$	$\frac{1}{2}$	$\frac{0}{2}$

Each row specifies the pmf for one document. In this example, we use the relative frequency of $Y = k$ given $X = x$ in a dataset of labelled documents.

Limitations of Tabular CPDs

A tabular CPD grows with:

[3] A Naive Bayes Classifier (NBC; Appendix B of textbook) is an example of a better way to approach text classification with tabular CPDs.

Limitations of Tabular CPDs

A tabular CPD grows with: the number of outcomes and conditions: one parameter for each possible class, per condition.

[3] A Naive Bayes Classifier (NBC; Appendix B of textbook) is an example of a better way to approach text classification with tabular CPDs.

Limitations of Tabular CPDs

A tabular CPD grows with: the number of outcomes and conditions: one parameter for each possible class, per condition.

Easy to estimate from data. For example, the maximum likelihood estimate is a table of relative frequencies.

- but it grows with the number of observed conditions;
- and the table is mostly sparse (like in our example);
- and unseen documents have no place in the table.

All in all, tabular CPDs are based on too crude ML/stats to be useful for text classification.[3]

[3] A Naive Bayes Classifier (NBC; Appendix B of textbook) is an example of a better way to approach text classification with tabular CPDs.

Text Representation

Key Limitation of our Basic Example

Consider two documents

I

The team won the championship after a late goal. Fans celebrated the victory as the team lifted the trophy.

II

The champions returned home to cheering crowds. The captain thanked fans as players paraded the trophy through the city.

With a basic tabular view of CPDs, our mechanism to parameterise the pmf $p(\cdot|x^{I})$ is completely independent of the mechanism to parameterise $p(\cdot|x^{II})$.

Key Limitation of our Basic Example

Consider two documents

I

The team won the championship after a late goal. Fans celebrated the victory as the team lifted the trophy.

II

The champions returned home to cheering crowds. The captain thanked fans as players paraded the trophy through the city.

With a basic tabular view of CPDs, our mechanism to parameterise the pmf $p(\cdot|x^{I})$ is completely independent of the mechanism to parameterise $p(\cdot|x^{II})$.

We need a view of x^{I} and x^{II} that reveals their relatedness.

Representation Space

Let's imagine a way, a **feature function** $\mathrm{h}(\cdot)$, to map any given text x to a point in a $\mathbb{R}^D$.

Each coordinate of the feature function captures an aspect of the text that's potentially relevant to the task.

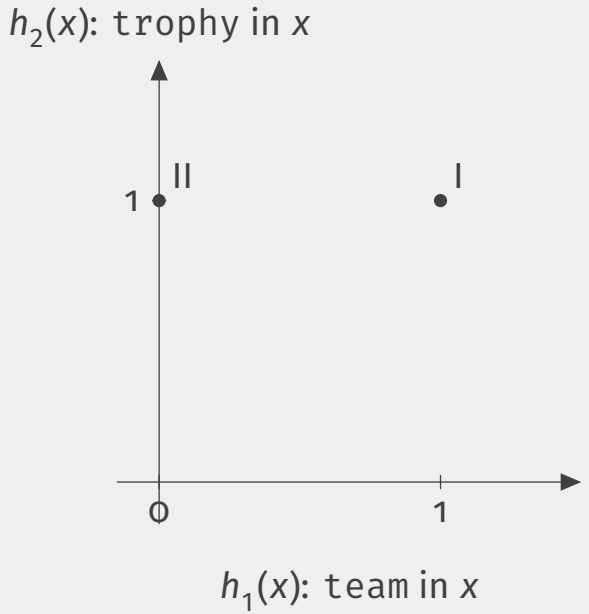
Representation Space

Let's imagine a way, a **feature function** $\mathbf{h}(\cdot)$, to map any given text x to a point in a $\mathbb{R}^D$.

Each coordinate of the feature function captures an aspect of the text that's potentially relevant to the task.

For example, words like 'team' and 'trophy' are relevant to sports. We can use dimensions of $\mathbf{h}(x)$ to indicate their presence in a tokenized view of x.

See tutorial 1 for tokenization.



A (sports)

The team won the championship after a late goal. Fans celebrated the victory as the team lifted the trophy.

B (tech)

The team launched a new chip for faster computing. The chip improves performance while reducing power consumption. The team demonstrated the technology.

Feature	$\mathbf{h}(A)$	$\mathbf{h}(B)$
championship		
computing		
late goal		
new chip		
power consumption		
team		
trophy		
victory		

1 = present; 0 = absent

The selection of features is a choice of the designer; once chosen, this is a fixed aspect of the design. **See tutorial 1.**

A (sports)

The team won the championship after a late goal. Fans celebrated the victory as the team lifted the trophy.

B (tech)

The team launched a new chip for faster computing. The chip improves performance while reducing power consumption. The team demonstrated the technology.

Feature	$\mathbf{h}(A)$	$\mathbf{h}(B)$
championship	1	0
computing	0	1
late goal	1	0
new chip	0	1
power consumption	0	1
team	1	1
trophy	1	0
victory	1	0

1 = present; 0 = absent

The selection of features is a choice of the designer; once chosen, this is a fixed aspect of the design. **See tutorial 1.**

Named Axes

A (sports)

The team won the championship after a late goal. Fans celebrated the victory as the team lifted the trophy.

B (tech)

The team launched a new chip for faster computing. The chip improves performance while reducing power consumption. The team demonstrated the technology.

Word Counts (or BoW)

Feature	$\mathrm{h}(A)$	$\mathrm{h}(B)$
championship		
chip		
computing		
fans		
goal		
launched		
performance		
power		
team		
technology		
trophy		
victory		
won		

Values = occurrence counts

The selection of features is a choice of the designer; once chosen, this is a fixed aspect of the design. **See tutorial 1.**

Word Counts (or BoW)

A (sports)

The team won the championship after a late goal. Fans celebrated the victory as the team lifted the trophy.

B (tech)

The team launched a new chip for faster computing. The chip improves performance while reducing power consumption. The team demonstrated the technology.

Feature	$\mathrm{h}(A)$	$\mathrm{h}(B)$
championship	1	0
chip	0	2
computing	0	1
fans	1	0
goal	1	0
launched	0	1
performance	0	1
power	0	1
team	2	2
technology	0	1
trophy	1	0
victory	1	0
won	1	0

Values = occurrence counts

The selection of features is a choice of the designer; once chosen, this is a fixed aspect of the design. **See tutorial 1.**

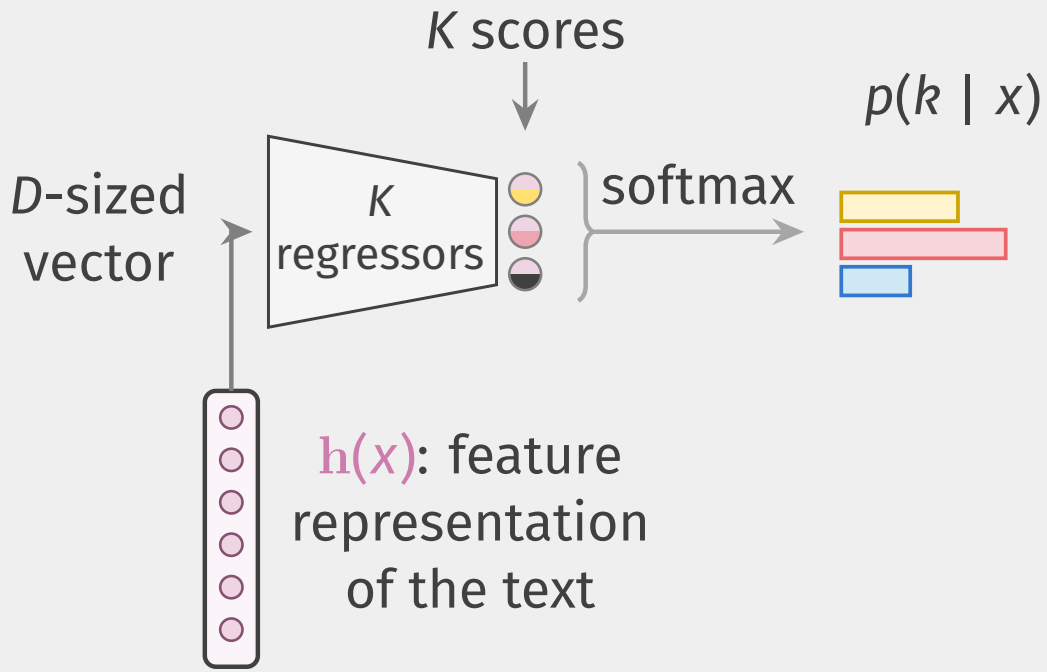
Typical Features

- Words that are frequent but not too frequent;
- Text normalisation (e.g., lowercasing, standardising punctuation, placeholders for numbers/date/time/URLs, etc); but note these things can also lose information;
- If words are complex and combinatorial, identify the root or stem and discard the rest (or at least give it special treatment);
- Multi-word expressions: consider using tuples of words as features (but mind combinational explosion);
  - Collocations: identify collocations (tokens that co-occur together more often than chance would predict);
  - Annotate words within the scope of certain function words (e.g., ‘not happy‘, ‘unhappy‘, ‘not really happy‘ could all become ‘NEGATION-happy‘).

Softmax Regression

Softmax Regression

A fixed representation $\mathbf{h}(x)$ of the input text x is mapped to K scores (or *logits*), which are then turned into a normalised probability vector via softmax:



Recall that softmax exponentiates each score, then divides the result by the sum of exponentiated scores.

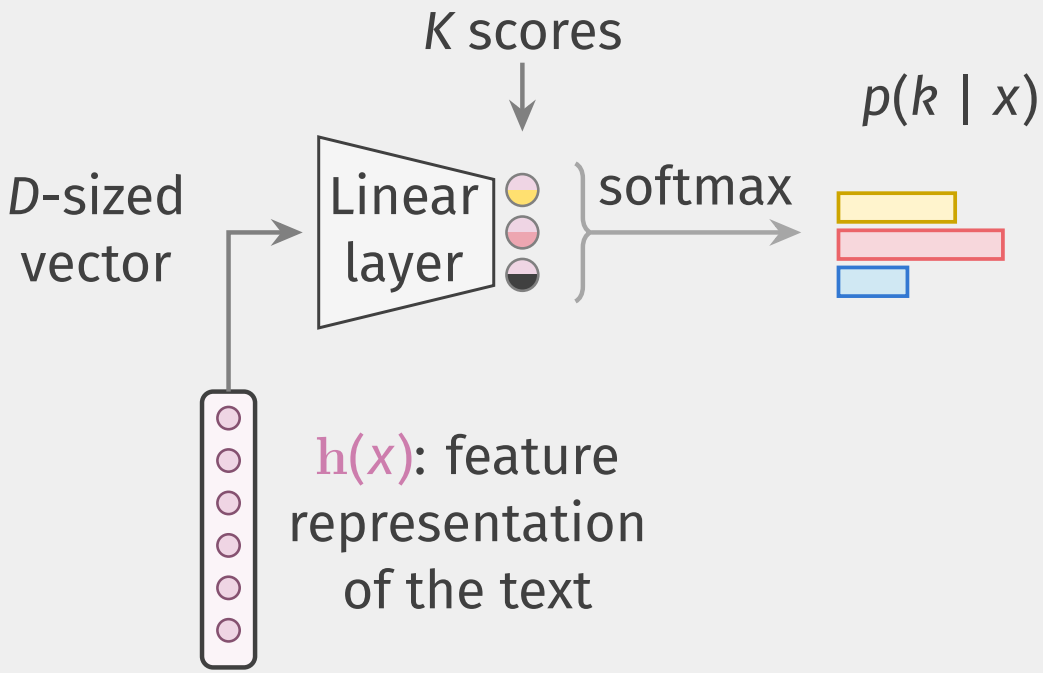
Softmax Regression

Log-Linear Parameterisation

Log-Linear Model

With *K linear regressors*, followed by softmax, we get a log-linear model. Each score $s_k(x)$ is obtained via

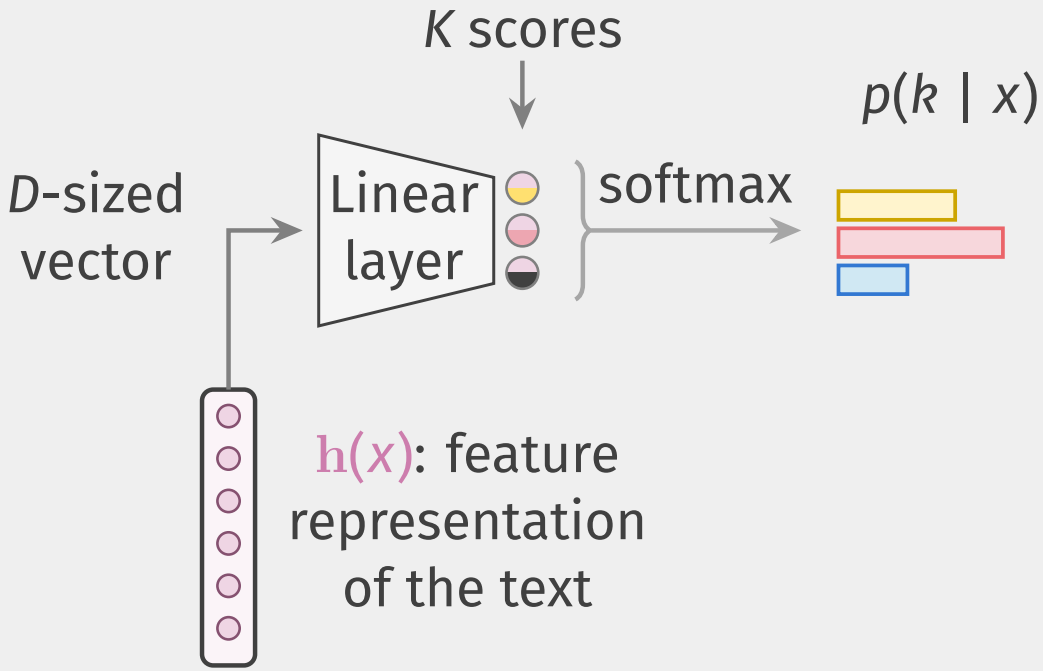
$$s_k(x) = \mathbf{w}_k^\mathsf{T}\mathbf{h}(x) + b_k \qquad \text{where } \mathbf{w}_k \in \mathbb{R}^D \text{ and } b \in \mathbb{R}$$



Log-Linear Model

With *K linear regressors*, followed by softmax, we get a log-linear model. Each score $s_k(x)$ is obtained via

$$s_k(x) = \mathbf{w}_k^\top \mathbf{h}(x) + b_k \qquad \text{where } \mathbf{w}_k \in \mathbb{R}^D \text{ and } b \in \mathbb{R}$$



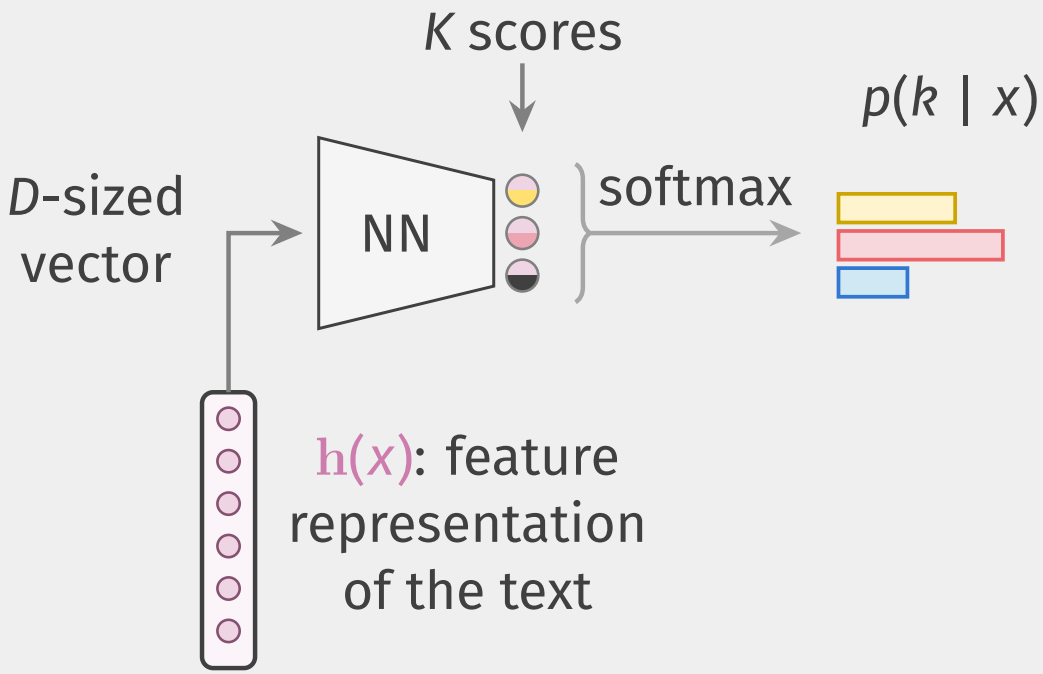
We have $D \times K + K$ parameters. They capture: affinity between features and classes ($w_{k,d}$); input-indep. bias for classes (b_k).

Softmax Regression

Neural Parameterisation

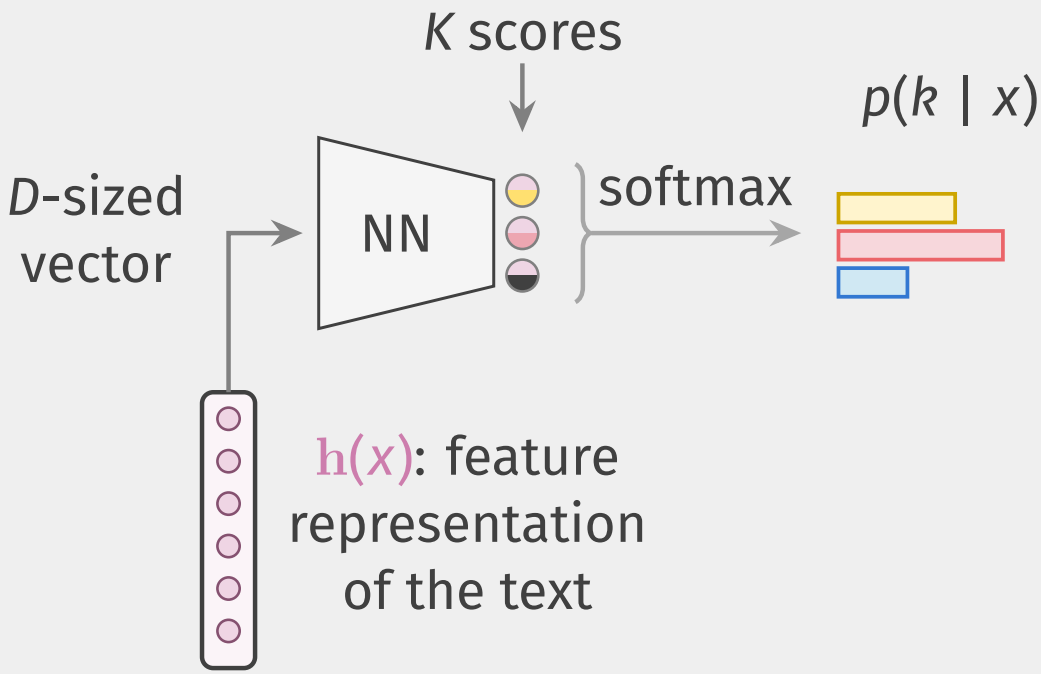
Neural Model

With a neural network for the regression component, we get a more expressive probabilistic model. In vector form, the K scores $\mathbf{s}(x)$ are obtained via $\mathbf{s}(x) = \text{NN}(\mathbf{h}(x); \theta)$:



Neural Model

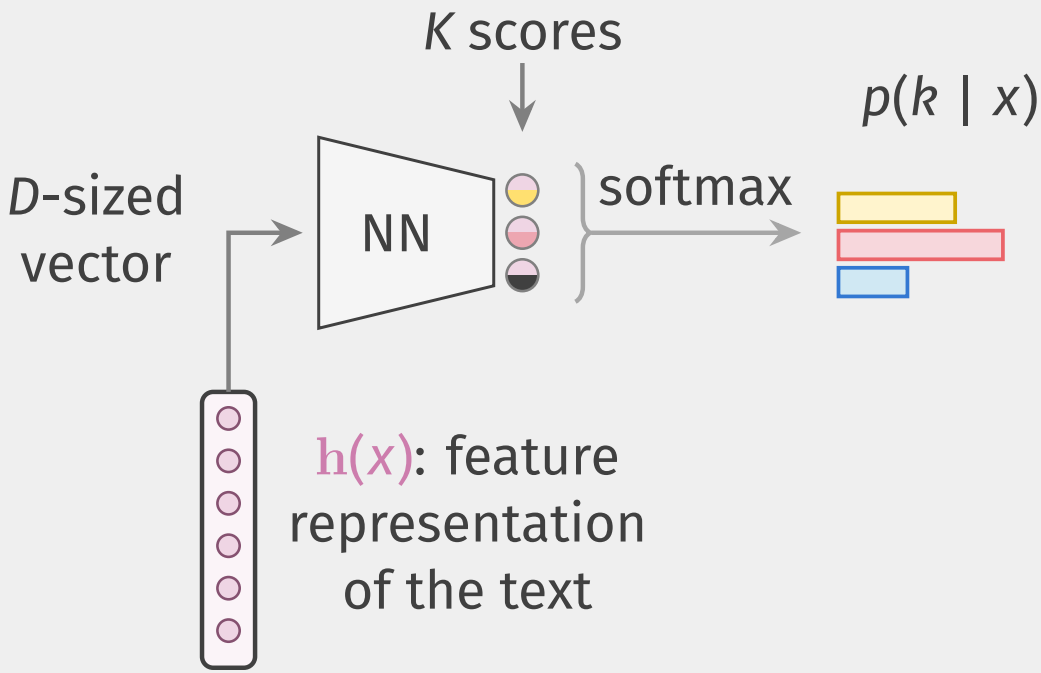
With a neural network for the regression component, we get a more expressive probabilistic model. In vector form, the K scores $\mathbf{s}(x)$ are obtained via $\mathbf{s}(x) = \text{NN}(\mathbf{h}(x); \theta)$:



A simple NN might be: ...

Neural Model

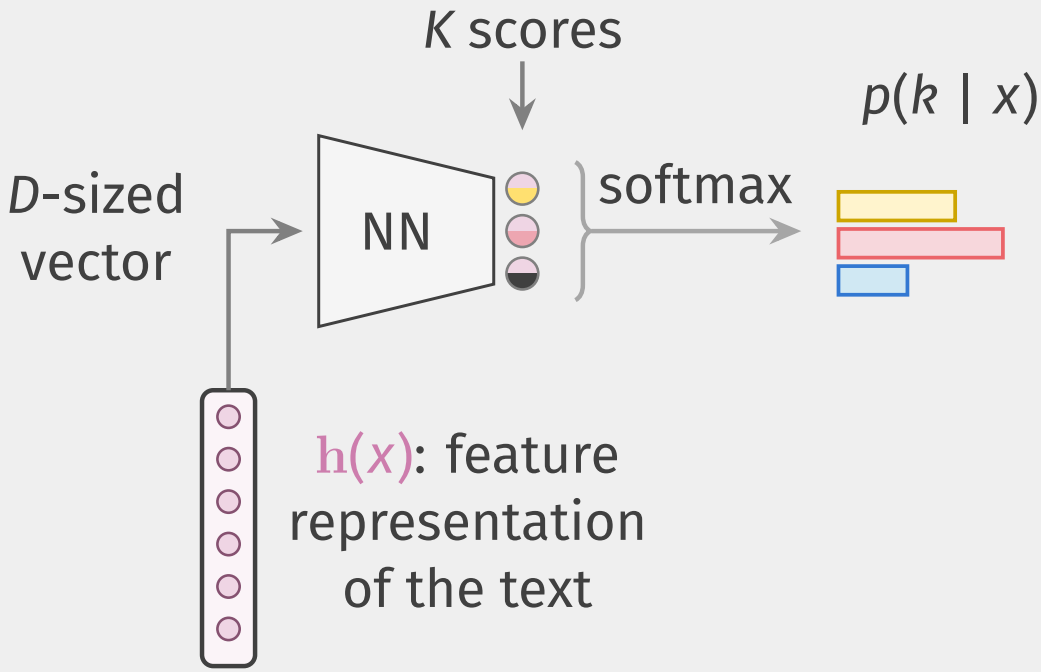
With a neural network for the regression component, we get a more expressive probabilistic model. In vector form, the K scores $\mathbf{s}(x)$ are obtained via $\mathbf{s}(x) = \text{NN}(\mathbf{h}(x); \theta)$:



A simple NN might be: $\text{linear}_K(\text{ReLU}(\text{linear}_H(\text{inputs})))$, then the parameters of the linear layers have shape ...

Neural Model

With a neural network for the regression component, we get a more expressive probabilistic model. In vector form, the K scores $\mathbf{s}(x)$ are obtained via $\mathbf{s}(x) = \text{NN}(\mathbf{h}(x); \theta)$:



A simple NN might be: $\text{linear}_K(\text{ReLU}(\text{linear}_H(\text{inputs})))$, then the parameters of the linear layers have shape $D \times H + H$ and $H \times K + K$.

Choosing an Architecture?

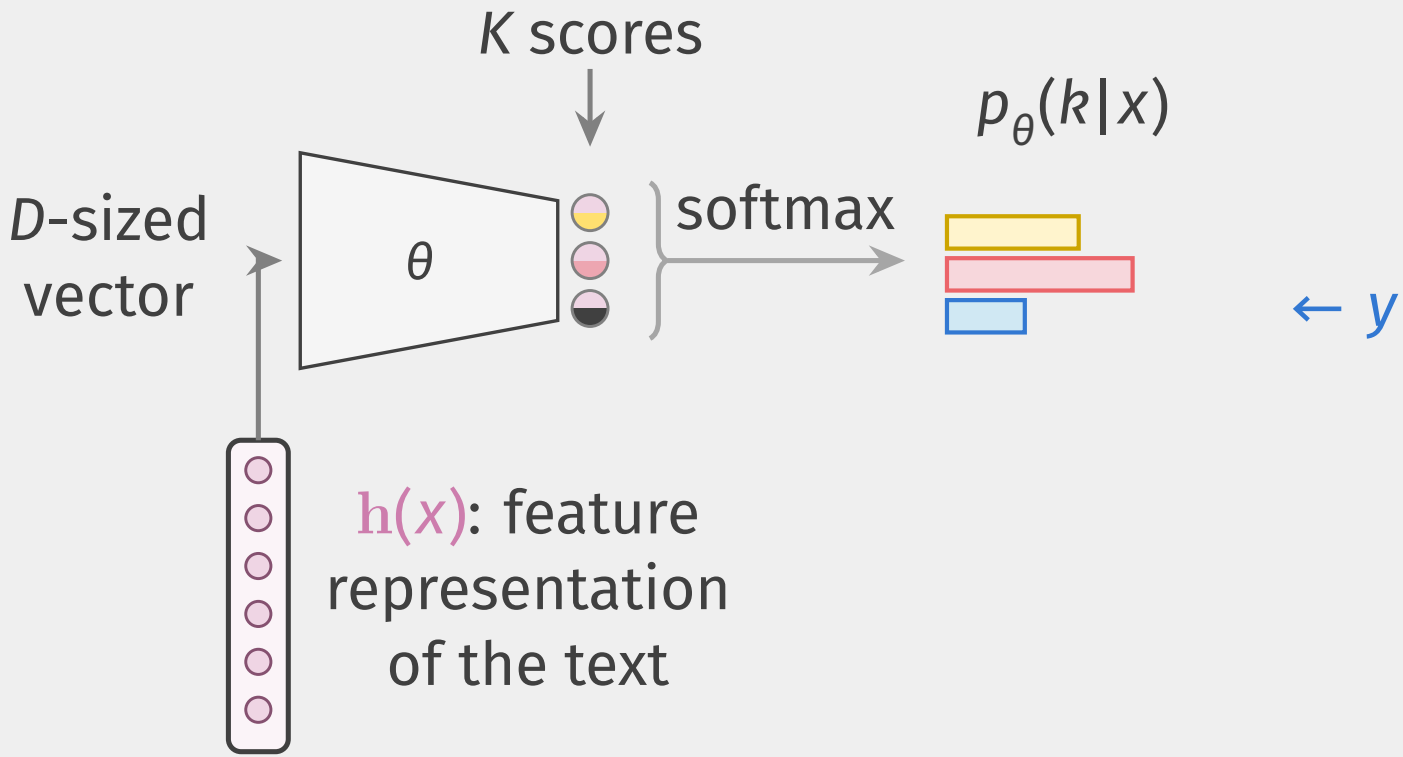
There's no real science here, you will have to try it out:

- As soon as you have 1 hidden layer, the model is able to combine input features in ways that might be beneficial.
- Theoretically, 1 wide-enough hidden layer can do whatever many hidden layers can do; in practice, width and depth are trade-offs for you to explore.
- Perhaps contrary to modern intuition: start simple.

Softmax Regression

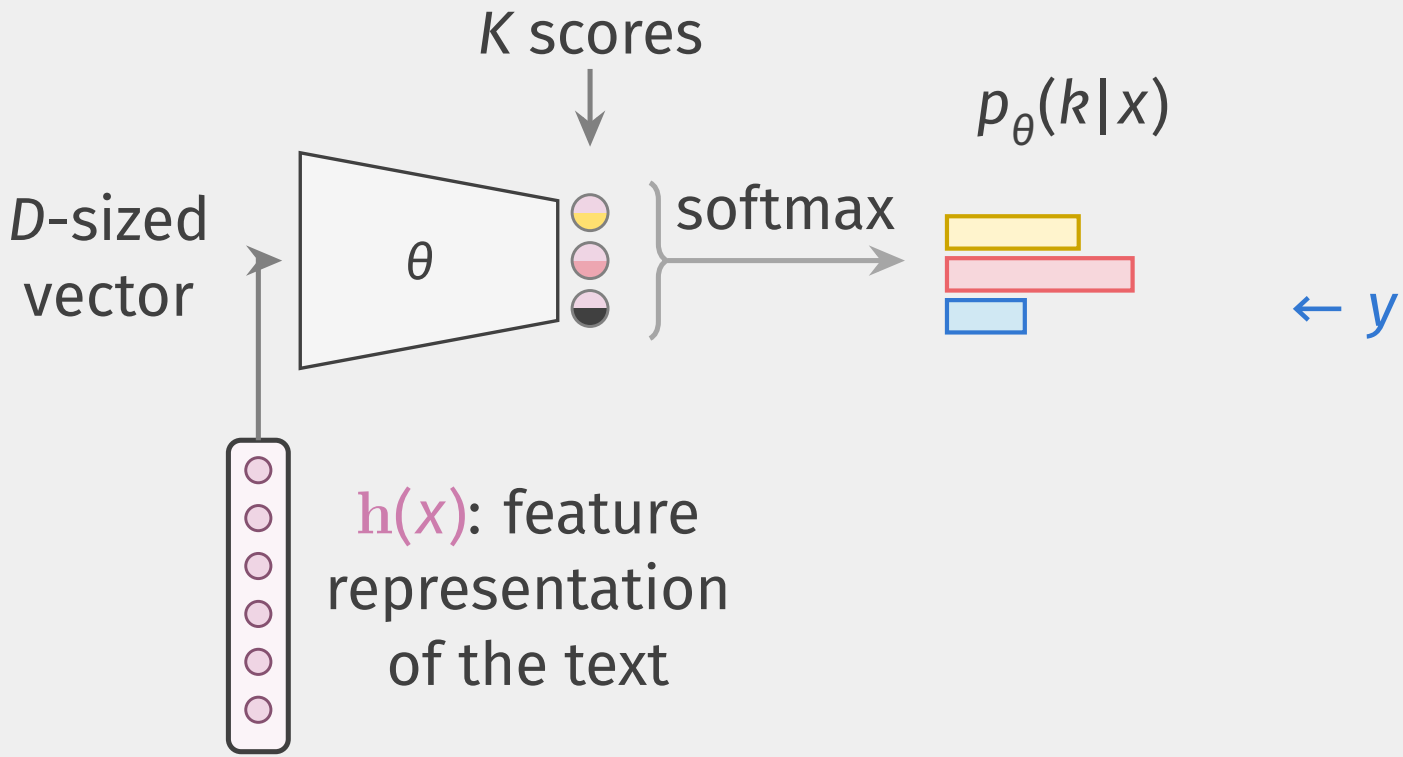
Parameter Estimation

Given an observation (x, y),



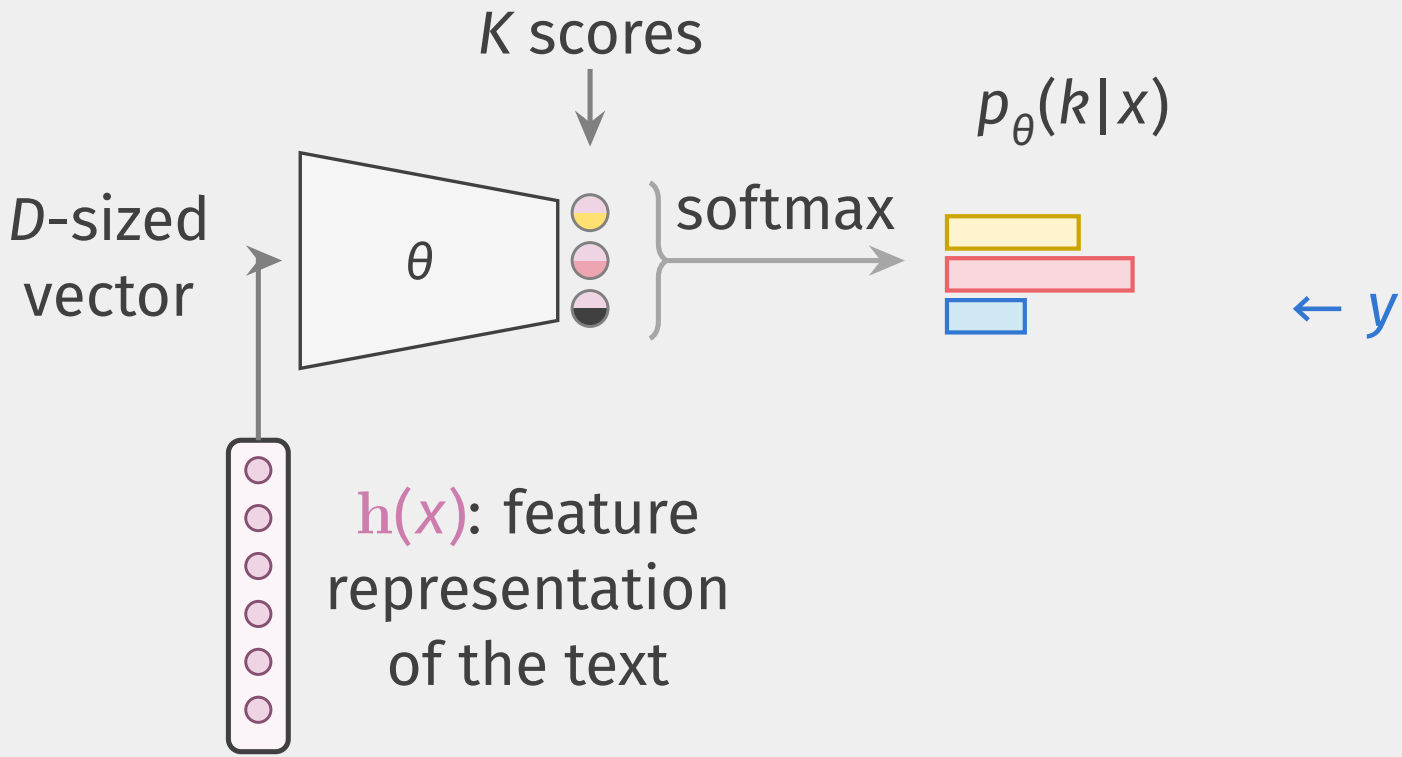
the current model assigns prob $p_\theta(y|x)$ to the observation.

Given an observation (x, y),



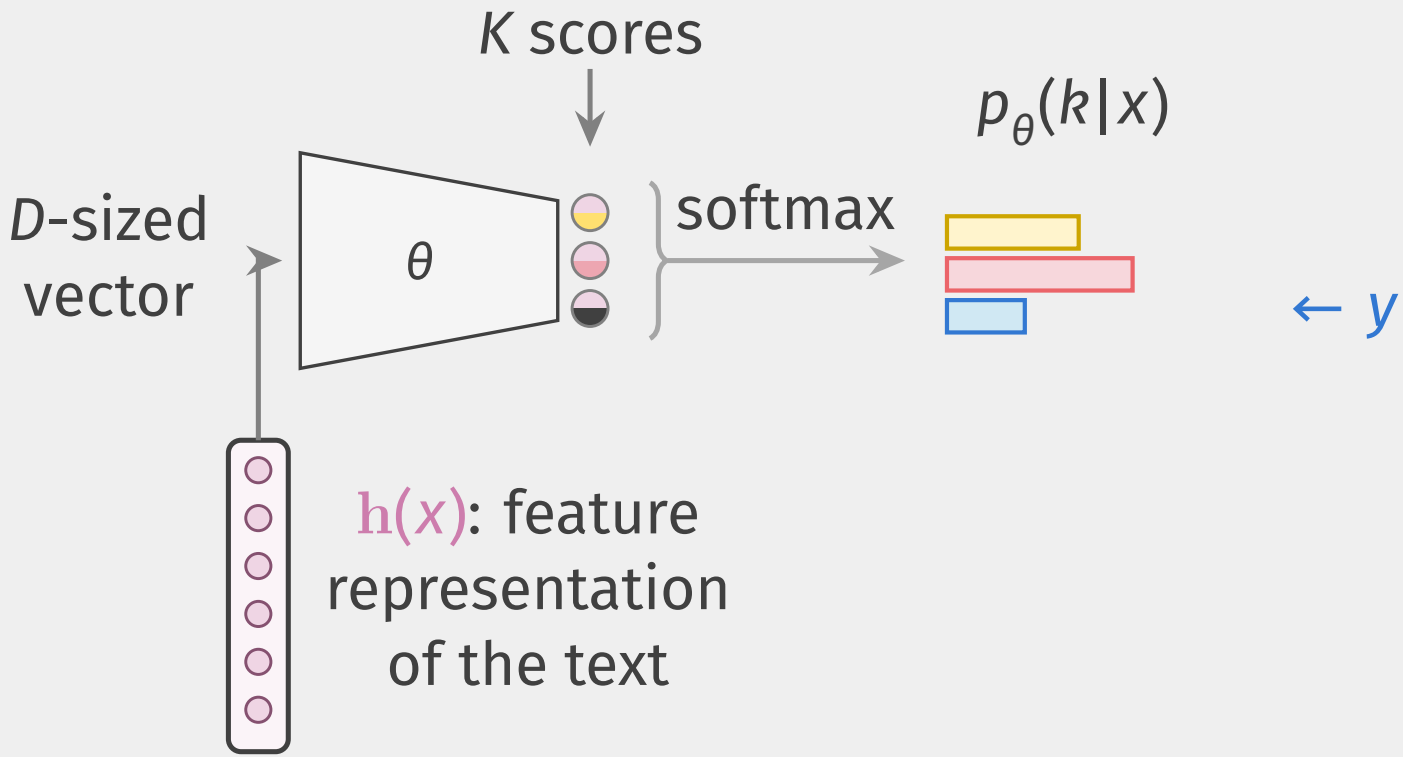
the quantity $\log p_\theta(y|x)$, when regarded as a function of θ, is a single-sample estimate of the model's log-likelihood.

Given an observation (x, y),



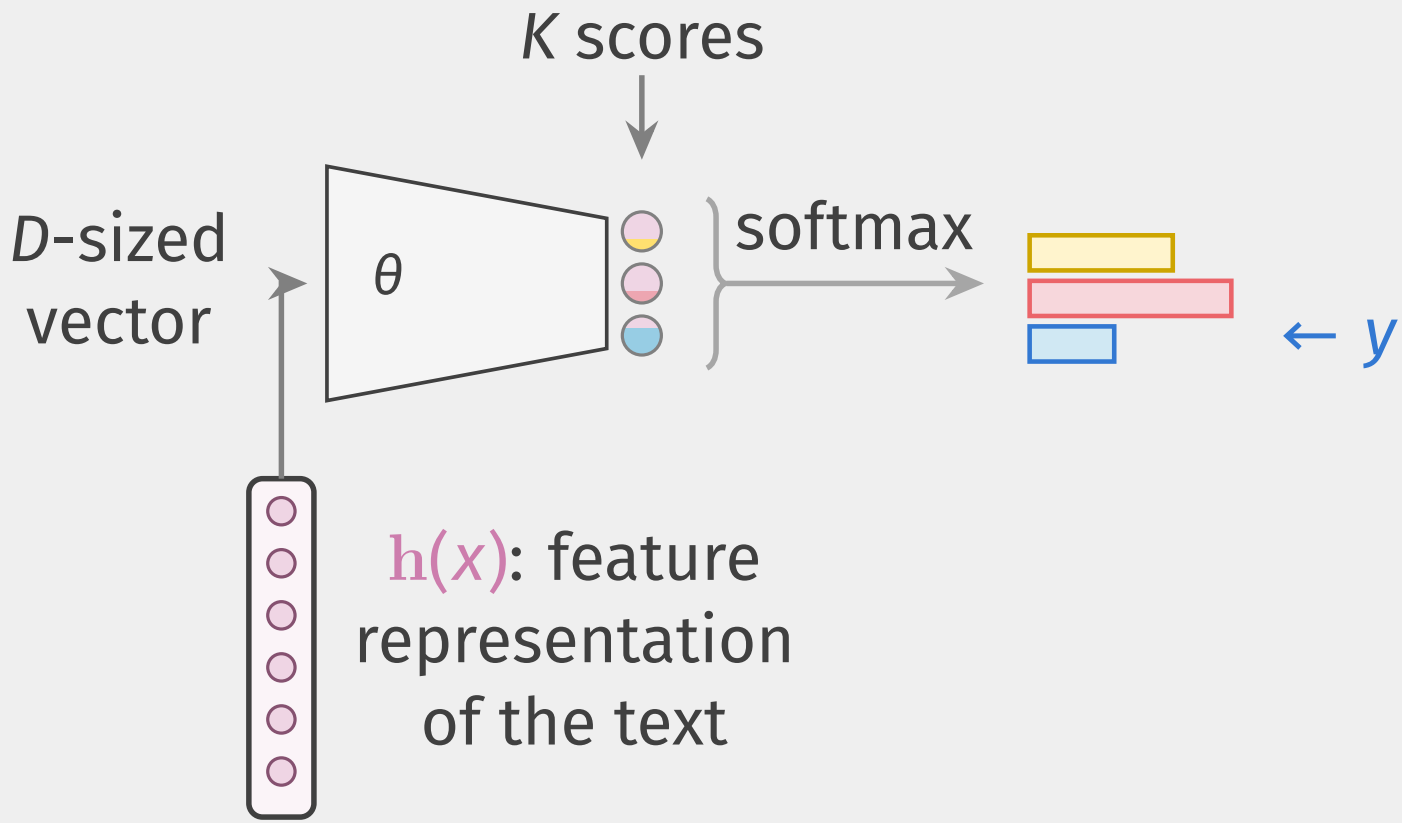
we can then characterise a *loss* for parameter estimation: $\ell(\theta|x, y) = -\log p_\theta(y|x)$.

Given an observation (x, y),



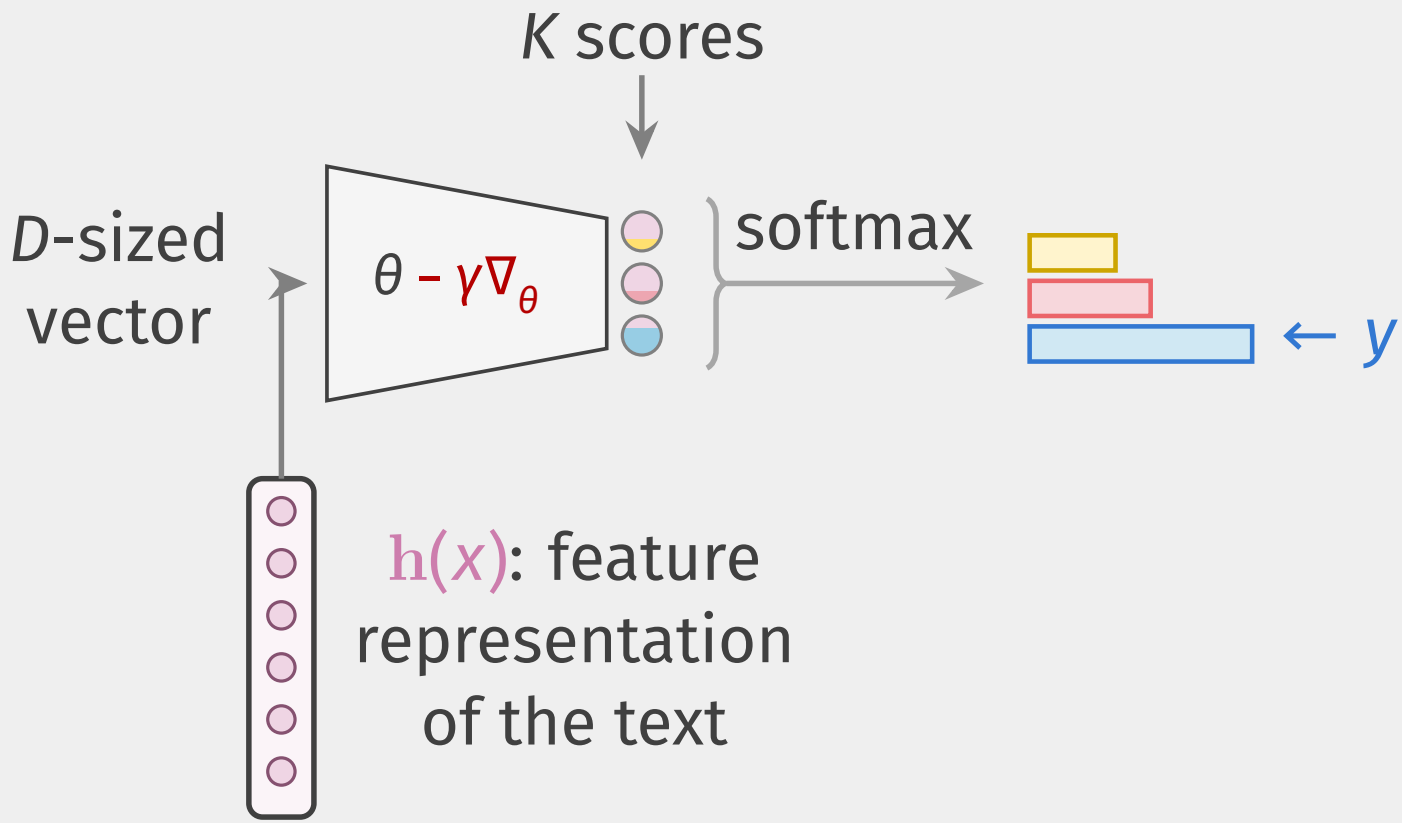
we want to choose θ so that $\ell(\theta|x, y)$ is minimal.

Given an observation (x, y),



An update in the direction of $-\nabla_\theta \ell(\theta|x, y)$ moves the model closer to a (local) optimum of the loss function.

Given an observation (x, y),



An update in the direction of $-\nabla_\theta \ell(\theta|x, y)$ moves the model closer to a (local) optimum of the loss function.

Strategy

Objective: maximise the model's likelihood

- we use log-likelihood, for numerical stability;
- we minimise its negative (aka cross-entropy loss);
- estimate the gradient using a subset (or batch) of observations (because it's more efficient and asymptotically equivalent);

Algorithm: iteratively, we

- assess the objective using the current parameters;
- calculate the gradient (via automatic differentiation);
- update the current parameters.

Modern optimisers maintain dynamic learning rates (important for convergence).

Regularisation

Weight decay: equivalent to L2 penalty on the weights.

Dropout: independently and at random, set a portion (e.g., 10%) of the inputs of linear layers to zero.

- used during training;
- test-time dropout has an approximate Bayesian interpretation, but it's not very common in NLP;[7]
- the dropout rate is a hyperparameter to be determined empirically (say via cross-validation).

[7] Optional (and advanced): dropout theory [3].

Evaluation

Evaluation Metrics

Probabilistic model: assess the training criterion with a different set of observations.

$$\frac{-1}{|T|} \sum_{x,y \in T} \log p_\theta(y|x)$$

Text classifier: compare model-based decisions to human-made decisions. [Textbook, Section 4.9]

- accuracy
- F1

Take Experimental Variance Seriously

Experiments vary for many reasons, incl. randomness in how we

- collect data; split datasets;
- initialise parameters; determine order of updates; etc.

We fix ‘random seeds’ so that we can document them and reproduce an experiment, not because ‘randomness’ is bad.

Don’t confuse ‘this result is reproducible’ (≈ this script spits the same results again and again) with this research ‘replicates’. The former is a not a proxy to the latter.

Cross-validation, multiple runs, data visualisation and statistics are tools to help us gain trust in our decisions and findings.[8]

[8] See tutorial 1 for examples of what you can do.

Limitations

Uninformative Representations

A (sports)

The report covers football, not technology.

B (tech)

The report covers technology, not football.

Both texts have identical bag-of-words representations, but concern different topics.

Uninformative Representations

A (sports)

The report covers football, not technology.

B (tech)

The report covers technology, not football.

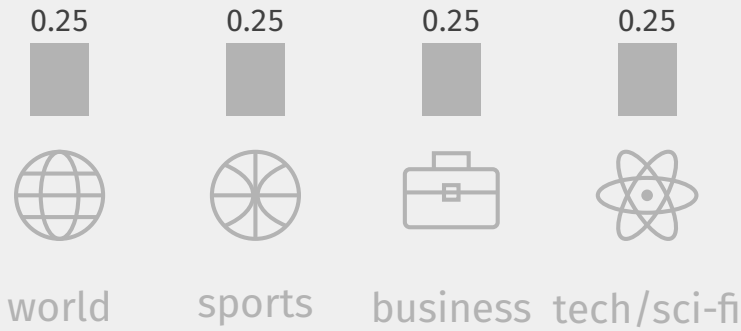
Both texts have identical bag-of-words representations, but concern different topics.

Better features via feature combination and engineering (e.g., n-gram features, mark words with negation scope, etc.) and/or better machine learning (next week).

Arbitrariness of Argmax



The home team won the league championship on Sunday after its captain scored in the final minute. Thousands of fans celebrated as the players lifted the trophy following a tense 2–1 victory.

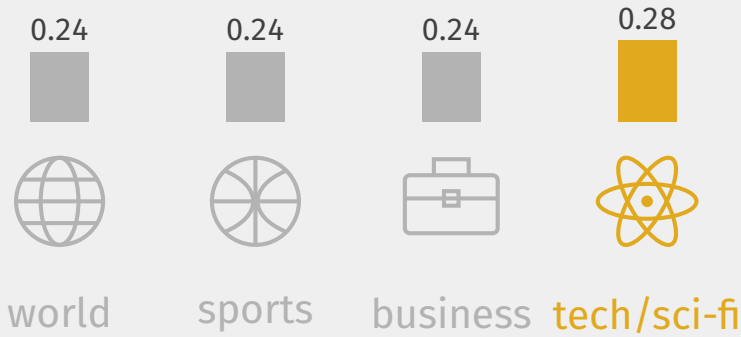


All four classes are equally probable: argmax needs a tie-breaking rule. Abstaining may be preferable, if allowed.

Arbitrariness of Argmax



The home team won the league championship on Sunday after its captain scored in the final minute. Thousands of fans celebrated as the players lifted the trophy following a tense 2–1 victory.



Argmax selects **tech/sci-fi**. But if we take the predicted pmf seriously, it assigns probability **0.72** to the label **not** being tech/sci-fi.

Uncertainty-Aware Decisions and Evaluation

Here are some strategies to counter limitations due to the **arbitrariness of argmax**.

Maximisation of expected utility: a decision rule that maximises the expectation of a user-defined utility function. See for example, Section 1.5 of Bishop's PRML or [2].

Uncertainty-aware evaluation: correlate confidence in prediction and correctness. [for example [4]]

- selective prediction
- AUROC

These are optional topics, for now.

Label Variation and Data Collection

Some tasks are such that, given an input, it can be argued that a single output is correct, out of the many that are possible. Example: mathematical or logical problems.

But some tasks (e.g., sentiment analysis) admit response variability. There are many reasons for that, for example:

- difference in perspectives
  is it okay if Claude co-authors my work?
  [what work? who is in the receiving end?]
- inputs that are under-specified
  how many votes did Lula get in the Brazilian election?
  [which year?]

If you are interested in these kinds of tasks specifically (and I'd argue most tasks are like that), optionally check [1].

Summary

- Text classifiers combine a probabilistic model and a decision rule;
- the model maps text to a pmf;
- the decision rule maps the pmf to an outcome;
- we can evaluate models for the pfms they output and/or for the decisions they support;
- good models rely on good text representations;
- some simple representations can go a long way, but they usually abstract away from linguistic complexity;
- better text representations require solving non-trivial NLP problems;

What Next?

Self-study

- Chapter 4 of textbook (skip 4.11 and treat 4.15 as optional)
  `https://web.stanford.edu/~jurafsky/slp3/4.pdf`
- Useful before next class: sections 14.1 and 14.2 of textbook
  `https://web.stanford.edu/~jurafsky/slp3/14.pdf`

Deadlines

- Groups should be formed by now.
- Chose your task/dataset.

Next class: Text generation

References I



Joris Baan, Raquel Fernández, Barbara Plank, and Wilker Aziz. **Interpreting predictive probabilities: Model confidence or human label variation?** In Yvette Graham and Matthew Purver, editors, *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 268–277, St. Julian's, Malta, March 2024. Association for Computational Linguistics.



Bryan Eikema and Wilker Aziz. **Sampling-based approximations to minimum Bayes risk decoding for neural machine translation.** In Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang, editors, *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 10978–10993, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics.

References II



Yarin Gal and Zoubin Ghahramani.
Dropout as a bayesian approximation: Representing model uncertainty in deep learning.
In *Proceedings of The 33rd International Conference on Machine Learning*, pages 1050–1059. PMLR, 2016.



Evgenia Ilia and Wilker Aziz.
A tale of two error categories: Exploring concealed trade-offs in the errors of automated judges in evaluation of uncertainty quantifiers.
In *Proceedings of the 19th International Conference on Natural Language Generation (INLG)*, 2026.
arXiv:2610.09693 [cs.AI].